CSE4007
Artificial Intelligence

Lecture 6: Gated Recurrent Neural Networks

Department of Data Science
Hanyang University

Outline

Challenge: Learning Long-Term Dependencies

Gated RNNs

- Long Short-Term Memory (LSTM)
- LSTM Variants

Summary

Outline

Challenge: Learning Long-Term Dependencies

Gated RNNs

- Long Short-Term Memory (LSTM)
- LSTM Variants

Summary

Challenges in RNN training

- If well trained, RNN can learn dependencies across hundreds of steps!
  - But very hard to train in practice

- Basic problem: gradients propagated over many stages tend to either
  - Vanish (most of the time) or
  - Explode (rarely, but with much damage to optimization)

- Difficulty with long-term dependencies arises from
  - Exponentially smaller weights given to long-term interactions

- Example: predicting the next word

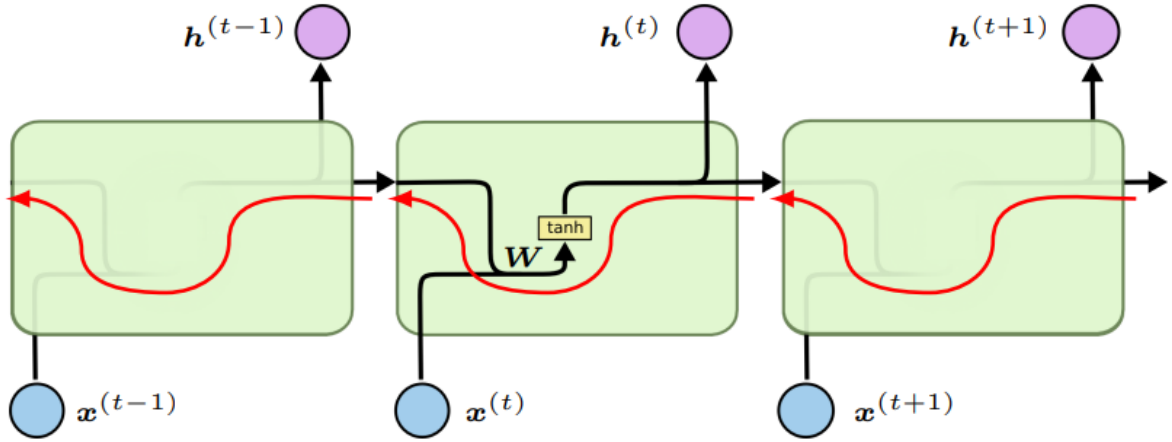
(a) Jane walked into the room. John walked in too. Jane said hi to _____.

(b) Jane walked into the room. John walked in too. It was late in the day, and everyone was walking home after a long day at work. Jane said hi to _____.

Gradient flow in vanilla RNN

- Backprop of gradient through the $\boldsymbol{h}$ path involves
  - Many factors of $\boldsymbol{W}$ and repeated tanh

$$\boldsymbol{h}^{(t)} = \tanh(\boldsymbol{W}_{hh}\boldsymbol{h}^{(t-1)} + \boldsymbol{W}_{hx}\boldsymbol{x}^{(t)} + \boldsymbol{b}_h)$$



- Repeated multiplication of the same $\boldsymbol{W}$
  - Largest singular value $< 1 \Rightarrow$ vanishing gradients
  - Largest singular value $> 1 \Rightarrow$ exploding gradients

Analyzing recurrence relation

- Consider recurrence relation (no activation/input for simplicity)

$$\boldsymbol{h}^{(t)} = \boldsymbol{W}^\top \boldsymbol{h}^{(t-1)}$$

  - Simplified to

$$\boldsymbol{h}^{(t)} = \left(\boldsymbol{W}^t\right)^\top \boldsymbol{h}^{(0)}$$

- If $\boldsymbol{W}$ admits an eigendecomposition (with orthogonal $\boldsymbol{Q}$)

$$\boldsymbol{W} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\top$$

  - the recurrence may be simplified further to

$$\boldsymbol{h}^{(t)} = \boldsymbol{Q}^\top \boldsymbol{\Lambda}^t \boldsymbol{Q}\boldsymbol{h}^{(0)}$$

Analyzing recurrence relation

- Each eigenvalue λ: raised to the power of t
  - If $|\lambda| < 1 \Rightarrow$ decay to zero
  - If $|\lambda| > 1 \Rightarrow$ explode

- This problem: particular to RNN

  (1) RNN: multiply the same weight $\boldsymbol{W}$ by itself many times
  - Product: $\boldsymbol{W}^t$

  $\Rightarrow$ vanish or explode depending on the magnitude of $\boldsymbol{W}$

  (2) non-RNN: use different $\boldsymbol{W}^{(t)}$ at each time step
  - Product: $\prod_t \boldsymbol{W}^{(t)}$

  $\Rightarrow$ careful scaling can avoid vanishing/exploding to some extent
  (remember the variance analysis for careful initialization)

Implications

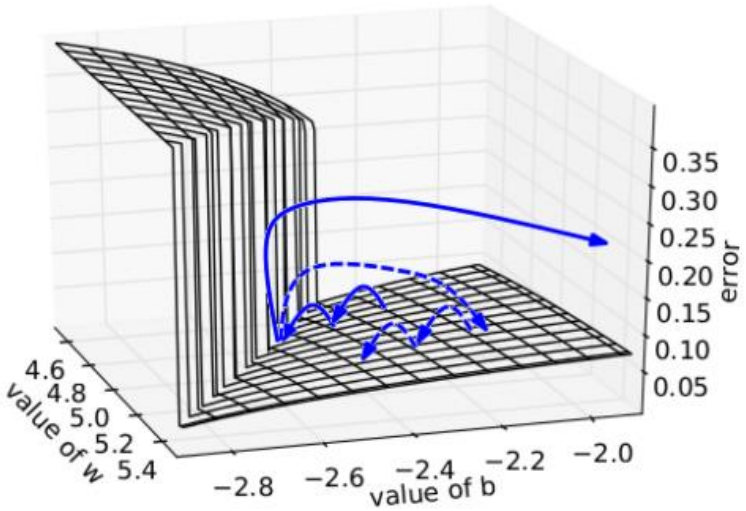
- Exploding gradient
  - Easy to detect ∵ overflow in gradient computation
  - A quick solution: gradient clipping

- Vanishing gradient
  - Can go undetected while drastically hurting training
    - Hurts both learning quality and speed
  - Heuristic solutions exist (e.g. IRNN)
  - More general solution: need to change RNN architecture

Gradient clipping

- A simple solution to exploding gradient
  - Clip gradients to a small number whenever they explode

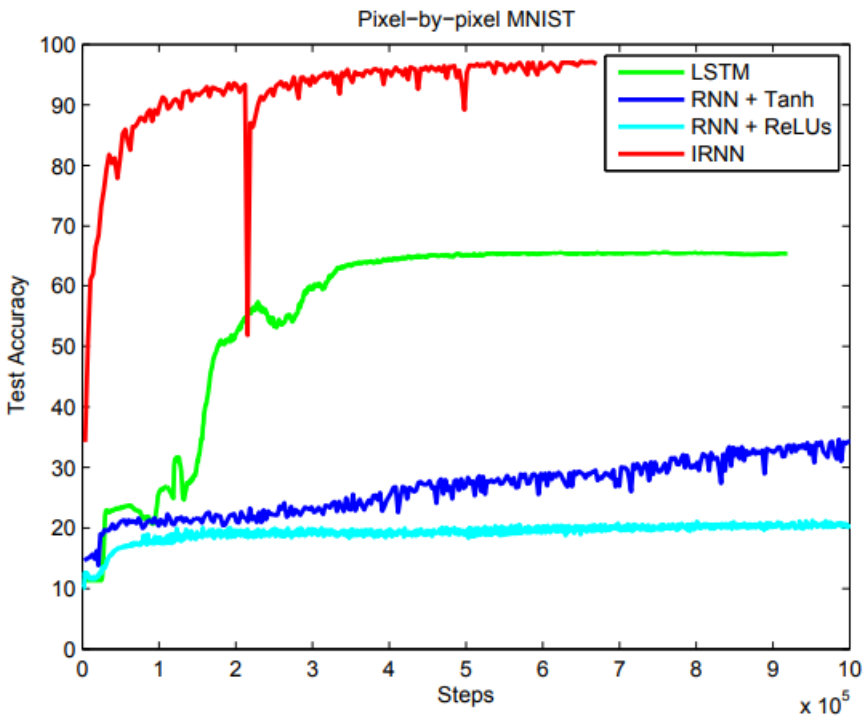
- Example: effect of gradient clipping
  - Error surface of a single hidden unit RNN
  - Solid lines: standard sgd
  - Dashes: sgd with clipping

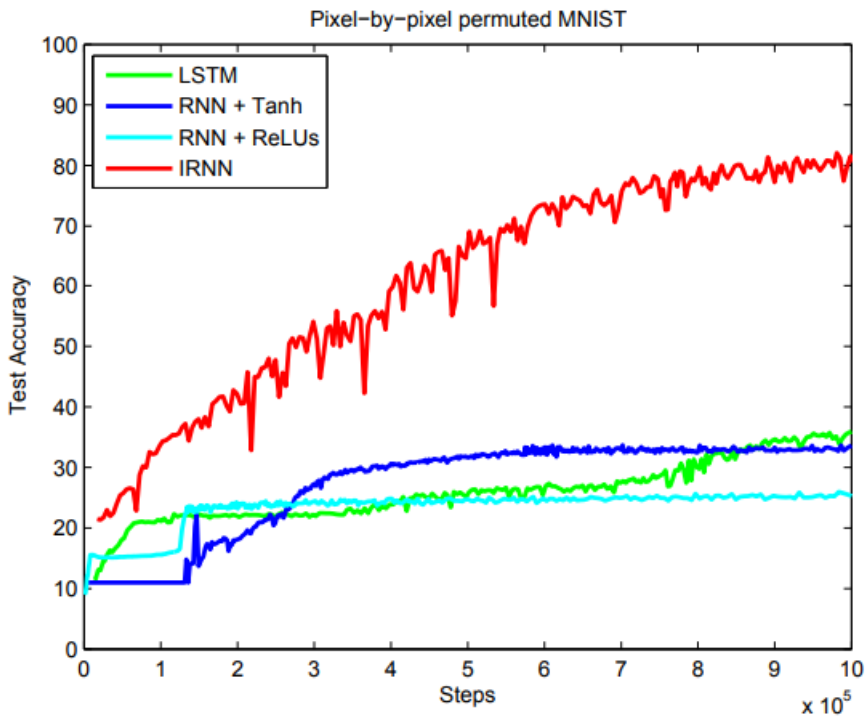


When the standard gradient descent model hits the high error wall, the gradient is pushed off to a faraway location on the decision surface; the clipping model instead pulls back the error gradient to somewhat close to the original gradient landscape.

IRNN

- A heuristic to handle vanishing gradient
  1. Initialize $\boldsymbol{W}$ to $\boldsymbol{I}$ (instead of randomly)
  2. Use ReLU (instead of Tanh)





Outline

Challenge: Learning Long-Term Dependencies

Gated RNNs

- Long Short-Term Memory (LSTM)
- LSTM Variants

Summary

Gated RNNs

- Very effective sequence models used in practical applications

  e.g. long short-term memory (LSTM)

  gated recurrent unit (GRU)

- Creates paths through time
  - These paths have derivatives that neither vanish nor explode
  - Connection weights may change at each time step
    - Not directly but through the action of gates

$$\underbrace{\boldsymbol{W}}_{\text{time-independent}} \rightarrow \underbrace{\text{gates}}_{\text{time-varying}} \rightarrow \text{gradient flow control}$$

$\Rightarrow$ no repeated multiplication by the same $\boldsymbol{W}$ anymore

Gated RNNs: Idea

- Use some units to allow the network to *accumulate* information
  - (e.g. evidence for a particular feature or category) over a long duration

- However, once that information has been used
  - It might be useful for the neural network to forget the old state

- Instead of manually deciding when to clear the state
  - We want the neural network to learn to decide when to do it

  ⇒ this what gated RNNs do!

Outline

Challenge: Learning Long-Term Dependencies

Gated RNNs

- Long Short-Term Memory (LSTM)
- LSTM Variants

Summary

LSTM

- Hochreiter and Schmidhuber, 1997
- A core contribution of initial LSTM:
  - Use self-loops to produce paths where gradient can flow for long durations

- A crucial addition: make weight on this self-loop
  - Conditioned on the context rather than fixed
  - "gated" (i.e. controlled by another hidden unit)
  - gating action changes dynamically (based on input)

- Extremely successful in many applications:
  - Handwriting recognition
  - Speech recognition
  - Handwriting generation
  - Machine translation
  - Image captioning
  - Many more!

LSTM networks

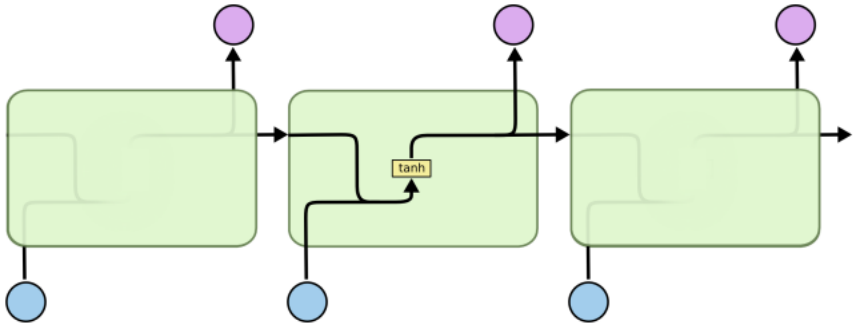
- Explicitly designed to avoid long-term dependency problem
  - Remembering info for long time: practically their default behaviour

- Standard RNNs: a chain of repeating modules

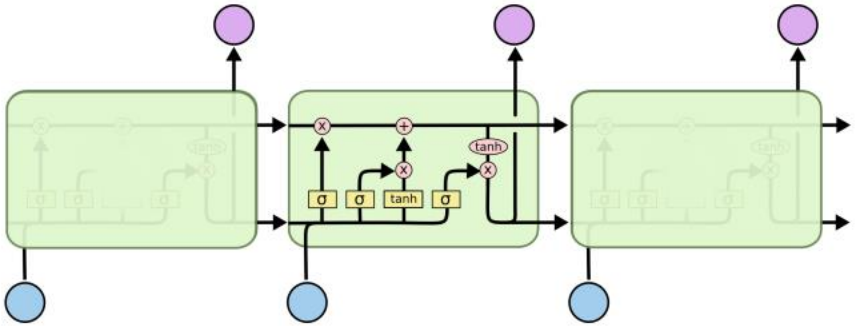
  Very simple structure (single tanh layer)

- LSTMs: also a chain of repeating modules

  more complex (four interacting network layer)



standard RNN



LSTM net

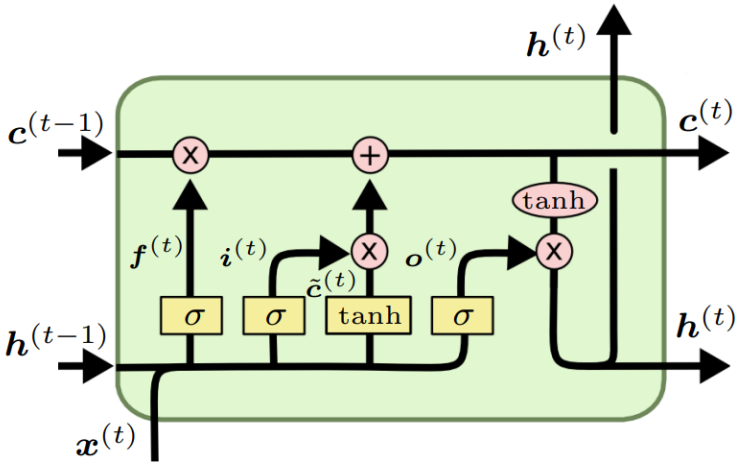
LSTM cells

- LSTM RNNs consists of "LSTM cells"
  - Have an internal recurrence (a self-loop) besides the outer recurrence
  - "nested recurrence"

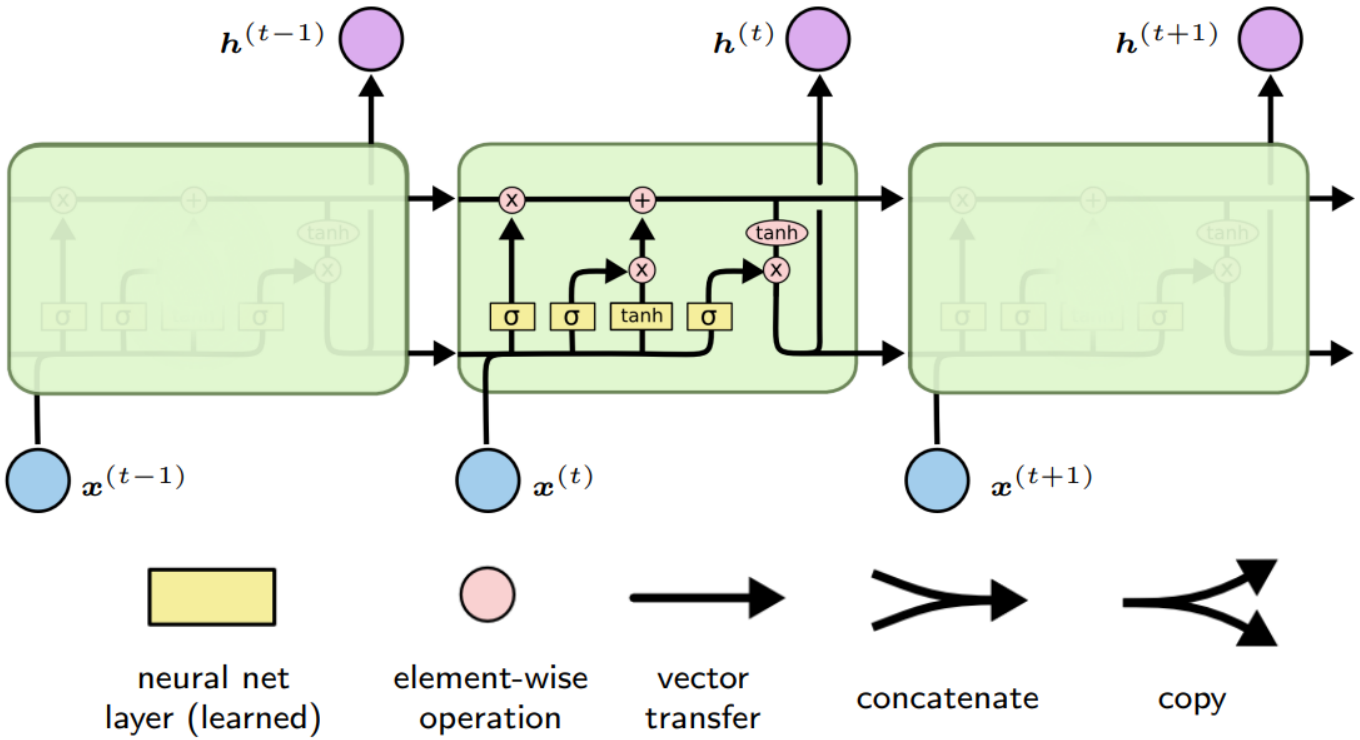
- Each LSTM cell: has the same inputs/outputs as in standard RNN
  - But has more parameters and a system of gating units

    Controls the flow of information

- Most important component
  - Cell state $\boldsymbol{c}^{(t)}$ (a vector!)
    - Internal recurrence



Notation and conventions



- Each line carries a vector
- $\sigma(\cdot)$: for controlling gates $(0 \leq \sigma \leq 1)$
- $\tanh(\cdot)$: for representing $(-1 \leq \tanh \leq 1)$
  - Hidden state (as in vanilla RNN) and increase/decrease in cell state

LSTM cells

- Simplified notation
  - Combine hidden-hidden and input-hidden weight matrices

$$\boldsymbol{h}^{(t)} = g(\boldsymbol{W}_{hh}\boldsymbol{h}^{(t-1)} + \boldsymbol{W}_{hx}\boldsymbol{x}^{(t)} + \boldsymbol{b}_h)$$
$$\triangleq g(\boldsymbol{W}_h[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_h)$$

where

$$\boldsymbol{W}_h \triangleq [\,\boldsymbol{W}_{hh} \quad \boldsymbol{W}_{hx}\,] \in \mathbb{R}^{n_h \times (n_h + n_x)}$$

$$[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] \triangleq \begin{bmatrix} \boldsymbol{h}^{(t-1)} \\ \boldsymbol{x}^{(t)} \end{bmatrix} \in \mathbb{R}^{(n_h + n_x) \times 1}$$

and

$$\boldsymbol{W}_h[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] = [\,\boldsymbol{W}_{hh} \quad \boldsymbol{W}_{hx}\,] \begin{bmatrix} \boldsymbol{h}^{(t-1)} \\ \boldsymbol{x}^{(t)} \end{bmatrix}$$
$$= \boldsymbol{W}_{hh}\boldsymbol{h}^{(t-1)} + \boldsymbol{W}_{hx}\boldsymbol{x}^{(t)}$$

Core idea behind LSTM

- Key: cell state
  - Like a conveyor belt
  - Runs down the entire chain (with only minor linear interactions)

  ⇒ information easily can just flow along the chain unchanged

- Each element in cell state

  ≈ scalar integer counter
  - Incremented/decremented

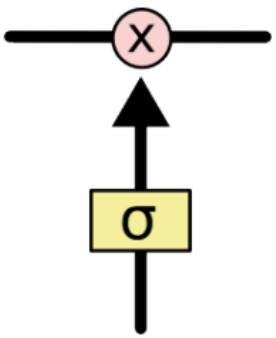
    by up to one

    at each time step

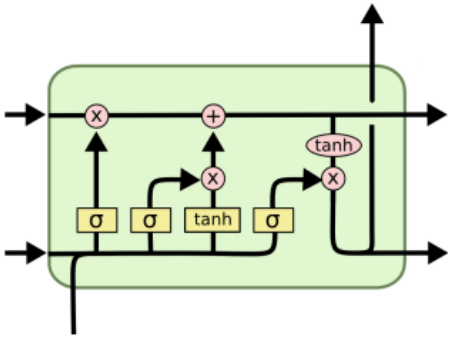
- LSTM can remove/add information to cell state
  - Carefully regulated by structures called **gates**

Core idea behind LSTM

- Gates: a way to optionally let information through
  - Composed of
    - A sigmoid neural network layer
    - An element-wise multiplication operation



$\Leftarrow$



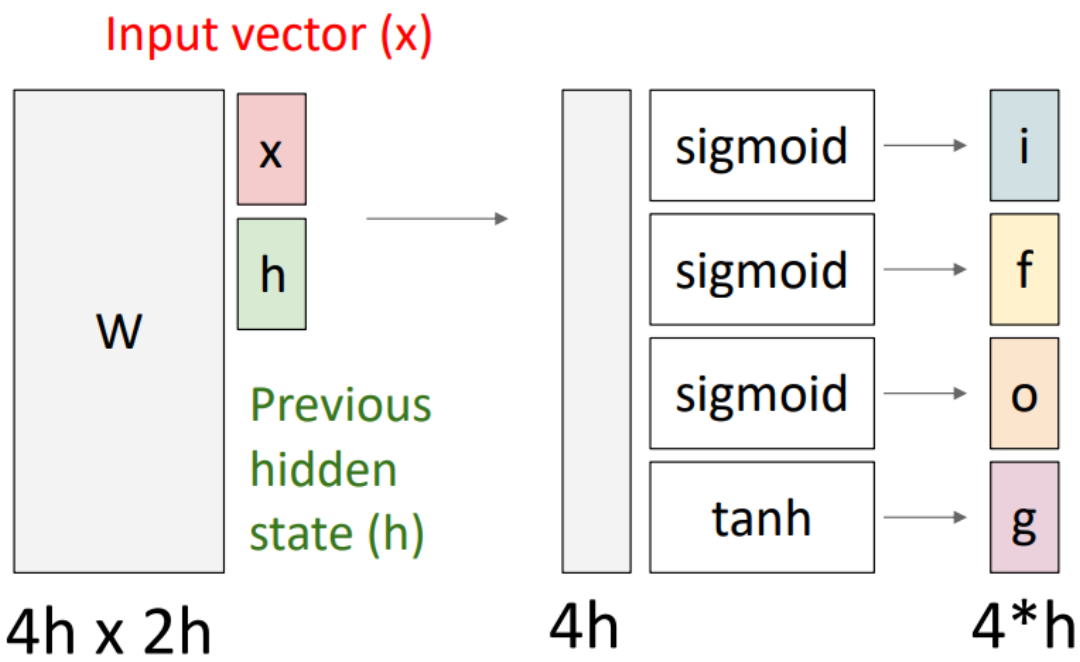
- The sigmoid layer
  - Describes how much of each element should be let through
  - Outputs a number between zero and one

$$\underbrace{0}_{\text{“let nothing through”}} \leq \text{output} \leq \underbrace{1}_{\text{“let everything through”}}$$

- Standard LSTM: has three gates (forget, input, output)

Core idea behind LSTM

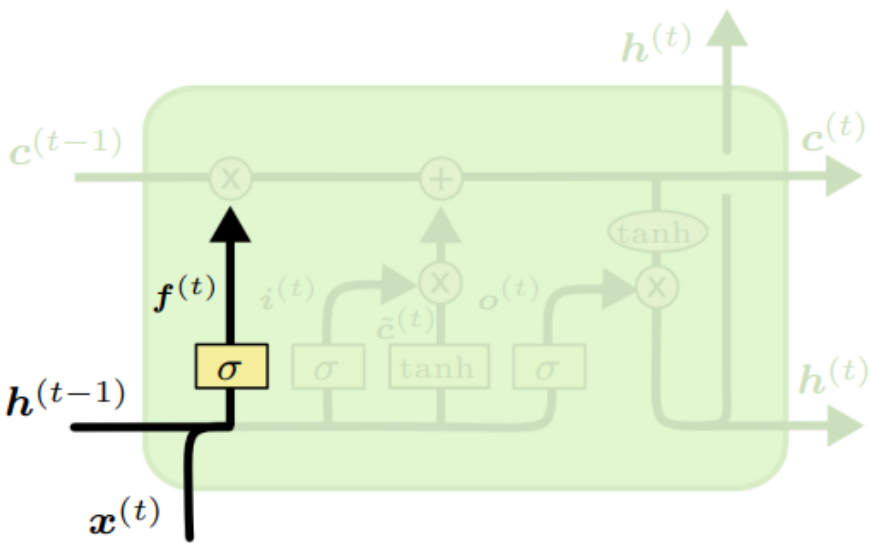
- **i**: <u>input gate</u>, whether to write to cell
- **f**: <u>forget gate</u>, where to erase cell
- **o**: <u>output gate</u>, how much to reveal cell
- **g**: <u>gate gate</u>, how much to write to cell



Step 1

- Decide what information to throw away from cell state
  - Implemented by a sigmoid layer called forget gate
  - Forget gate: for each number in cell state $\boldsymbol{c}^{(t-1)}$
    - Looks at $\boldsymbol{h}^{(t-1)}$ and $\boldsymbol{x}^{(t)}$
    - Outputs a number between zero and one

$$\underbrace{0}_{\text{"completely get rid of"}} \leq \text{output} \leq \underbrace{1}_{\text{"compeletely keep"}}$$



$$\boldsymbol{f}^{(t)} = \sigma(\boldsymbol{W}_f[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_f)$$

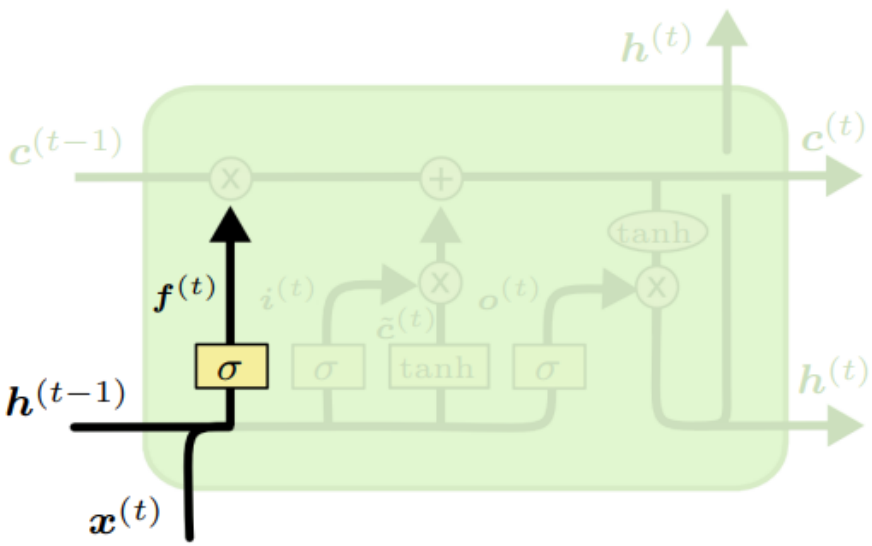
$$0 \leq \boldsymbol{f} \leq 1 \qquad \text{(gate)}$$

Example: language modeling

- Predict the next word based on all the previous ones

- Cell state
  - Might include the gender of the present subject
  - So that the correct pronouns can be used

- When we see a new subject
  - We want to forget the gender of the old subject



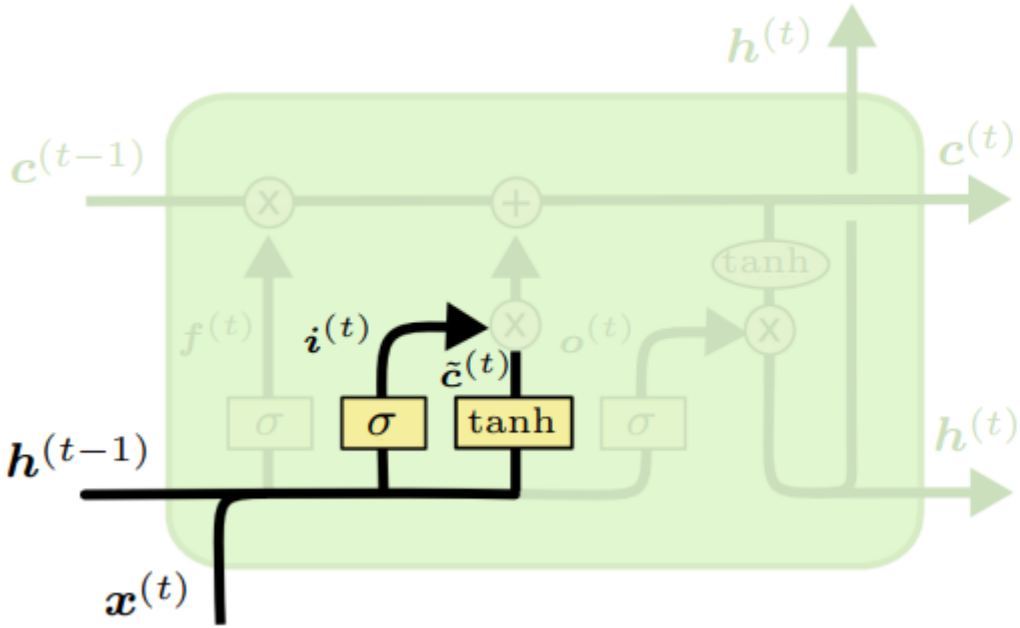
$$\boldsymbol{f}^{(t)} = \sigma(\boldsymbol{W}_f[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_f)$$

$$0 \leq \boldsymbol{f} \leq 1 \qquad \text{(gate)}$$

Step 2

- Decide what new information to store in cell state in two substeps
  1. A sigmoid layer (called input gate) decides which values to update
  2. A tanh layer creates a vector of $\underbrace{\text{new candidate values}}_{\text{Called } \tilde{\boldsymbol{c}}^{(t)}\text{, which could be added to the state}}$

※ step 3 will combine these two to create an update to the state



$$\boldsymbol{i}^{(t)} = \sigma(\boldsymbol{W}_i[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_i)$$

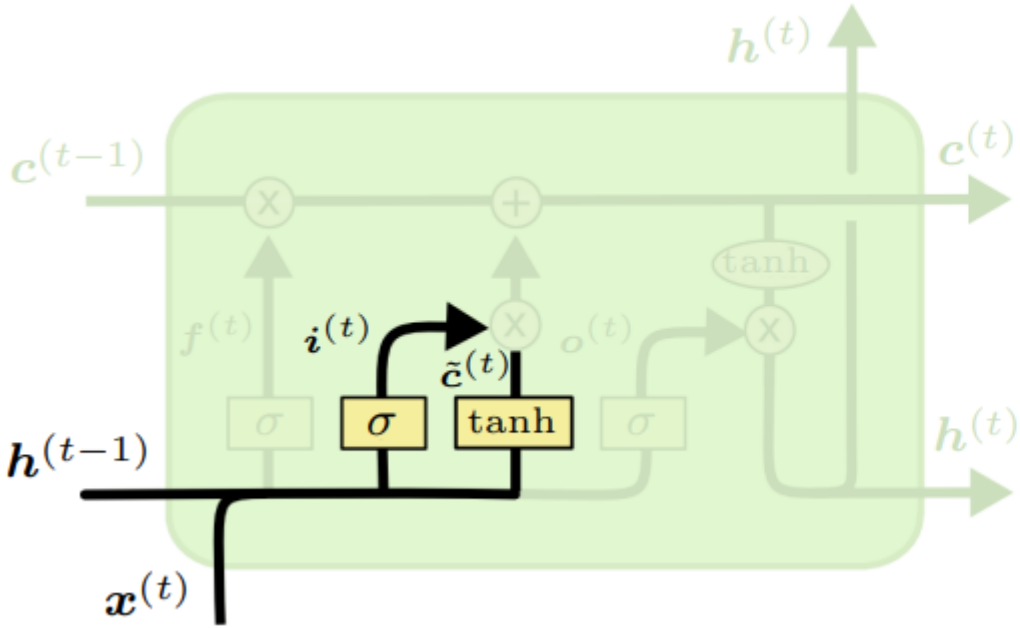
$$\tilde{\boldsymbol{c}}^{(t)} = \tanh(\boldsymbol{W}_c[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_c)$$

$$0 \leq \boldsymbol{i} \leq 1 \qquad \text{(gate)}$$

$$-1 \leq \tilde{\boldsymbol{c}} \leq 1 \qquad \text{(in/decrease)}$$

Example: language modeling (continued)

- We would want to add the gender of the new subject to cell state
  - To replace the old one we are forgetting



$$\boldsymbol{i}^{(t)} = \sigma(\boldsymbol{W}_i[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_i)$$

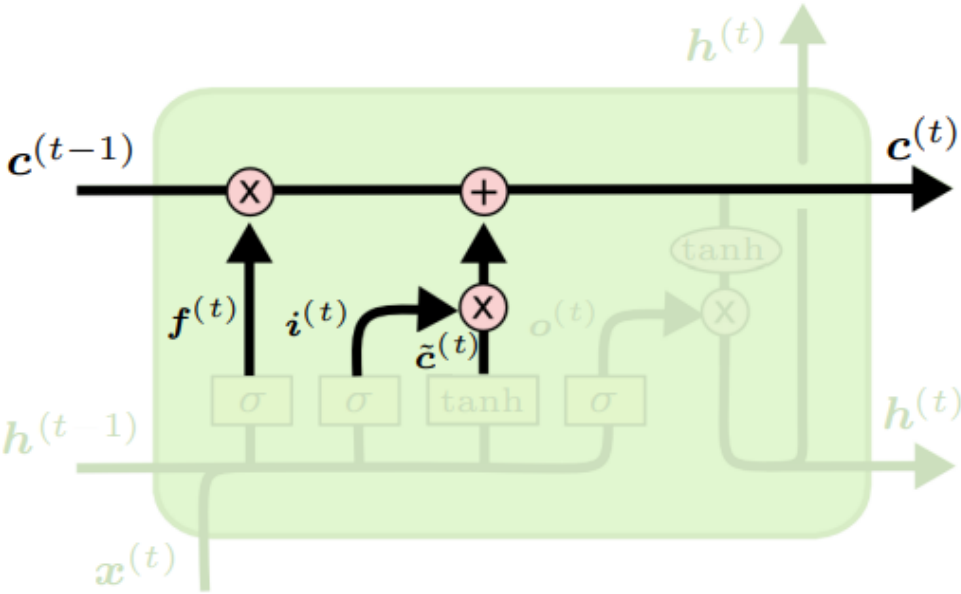
$$\tilde{\boldsymbol{c}}^{(t)} = \tanh(\boldsymbol{W}_c[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_c)$$

$$0 \leq \boldsymbol{i} \leq 1 \qquad \text{(gate)}$$

$$-1 \leq \tilde{\boldsymbol{c}} \leq 1 \qquad \text{(in/decrease)}$$

Step 3

- Now it's time to update old cell state $\boldsymbol{c}^{(t-1)}$ into new cell state $\boldsymbol{c}^{(t)}$
  - step 2 already decided what to do → we just need to actually do it now



$$\underbrace{\boldsymbol{c}^{(t)}}_{\text{new state}} = \overbrace{\boldsymbol{f}^{(t)}}^{\text{how much to forget}} \odot \underbrace{\boldsymbol{c}^{(t-1)}}_{\text{old state}} + \overbrace{\boldsymbol{i}^{(t)}}^{\text{how much to update}} \odot \underbrace{\tilde{\boldsymbol{c}}^{(t)}}_{\text{new candidate values}}$$

$$0 \leq \boldsymbol{f}, \boldsymbol{i} \leq 1 \qquad \text{(gates)}$$

$$-1 \leq \tilde{\boldsymbol{c}} \leq 1 \qquad \text{(in/decrease)}$$

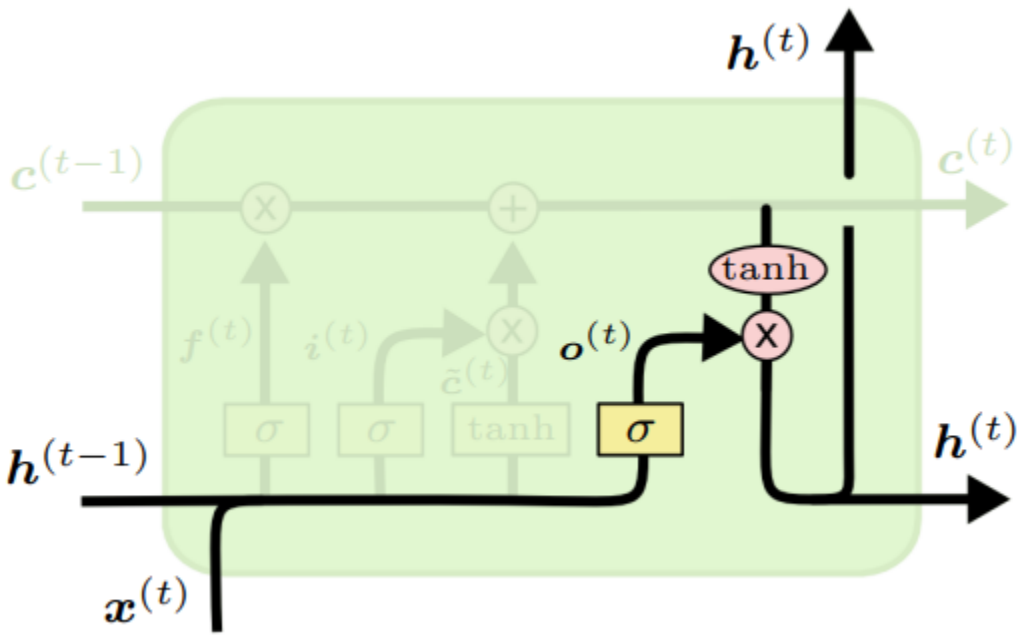
<u>Example: language modeling (continued)</u>

- We would actually drop the info about the old subject's gender
  - And add the new info as decided in step 2

Step 4

- Decide what to output ← based on cell state, but will be a filtered version
  1. Run a sigmoid layer (called output gate) to decide what parts of cell state to output
  2. Put cell state through tanh and multiply it by output gate

  ⇒ we only output the parts we decided to



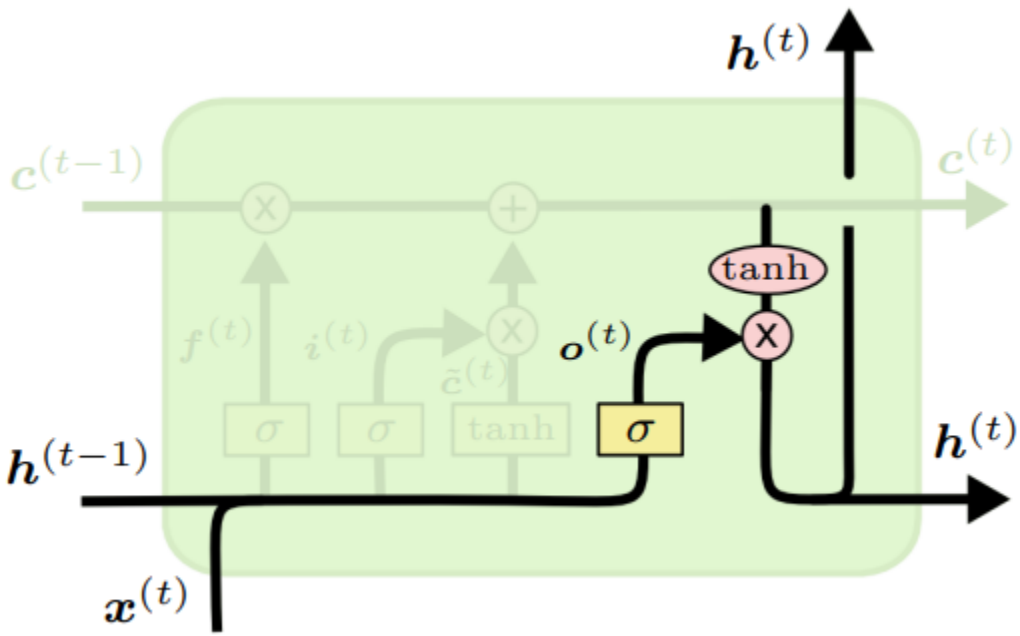
$$o^{(t)} = \sigma(W_o[h^{(t-1)}, x^{(t)}] + b_o)$$
$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

$$0 \leq o \leq 1 \qquad \text{(gate)}$$
$$-1 \leq h \leq 1 \qquad \text{(hidden state)}$$

Example: language modeling (continued)

- The model just saw a subject
  - It might want to output info relevant to a matching verb

  e.g. the subject is singular/plural



$$\boldsymbol{o}^{(t)} = \sigma(\boldsymbol{W}_o[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_o)$$

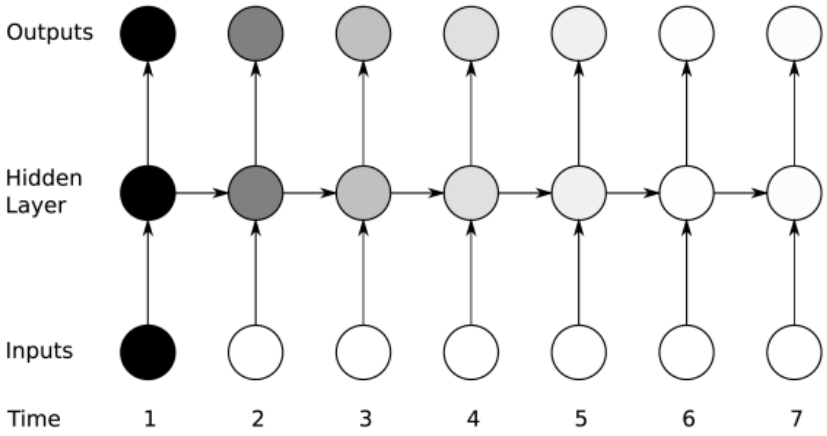
$$\boldsymbol{h}^{(t)} = \boldsymbol{o}^{(t)} \odot \tanh(\boldsymbol{c}^{(t)})$$

$$0 \leq \boldsymbol{o} \leq 1 \qquad \text{(gate)}$$

$$-1 \leq \boldsymbol{h} \leq 1 \qquad \text{(hidden state)}$$

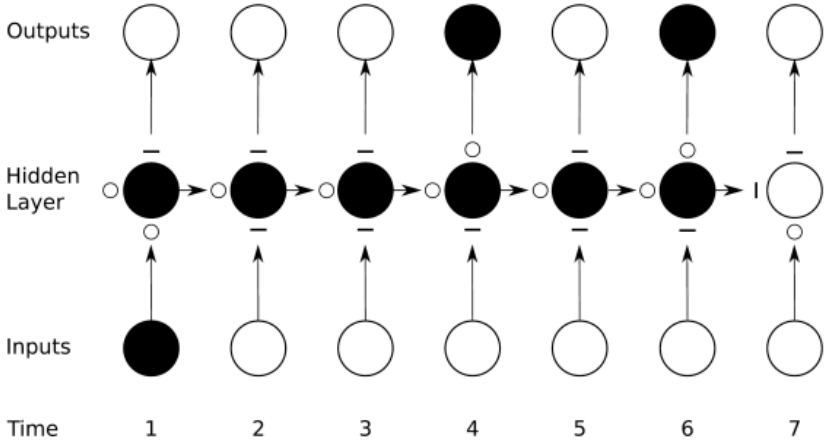
Comparison

- vanishing gradient (RNN)



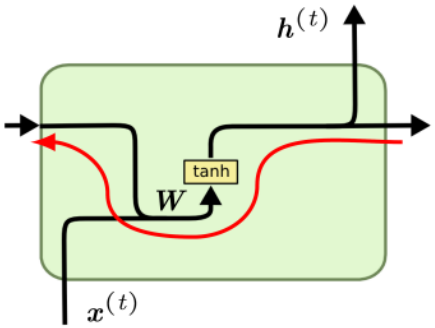
The shading of the nodes in the unfolded network indicates their sensitivity to the inputs at time one (the darker the shade, the greater the sensitivity). The sensitivity decays over time as new inputs overwrite the activations of the hidden layer, and the network forgets the first inputs.

- preservation of gradient (LSTM)



The black nodes are maximally sensitive and the white nodes are entirely insensitive. The state of the input, forget, and output gates are displayed below, to the left and above the hidden layer respectively. For simplicity, all gates are either entirely open (0) or closed (–). The memory cell 'remembers' the first input as long as the forget gate is open and the input gate is closed. The sensitivity of the output layer can be switched on and off by the output gate without affecting the cell.

Gradient flow in LSTM



c.f. vanilla RNN

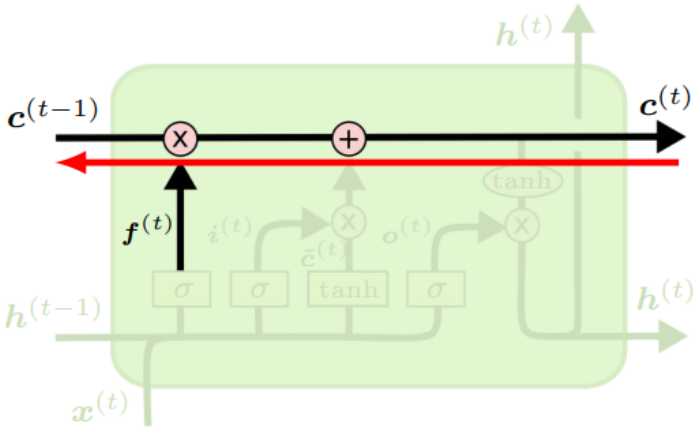
- Backpropagation from $\boldsymbol{c}^{(t)}$ to $\boldsymbol{c}^{(t-1)}$ involves
  - Only element-wise multiplication by $\boldsymbol{f}^{(t)}$
  - no matrix multiplication by $\boldsymbol{W}$

  ⇒ we only output the parts we decided to

- Much nicer than standard RNN for two reasons
  1. Element-wise multiplication: more efficient than full matrix multiplication
  2. Multiplication by forget gate
     - Potentially different values at every time step

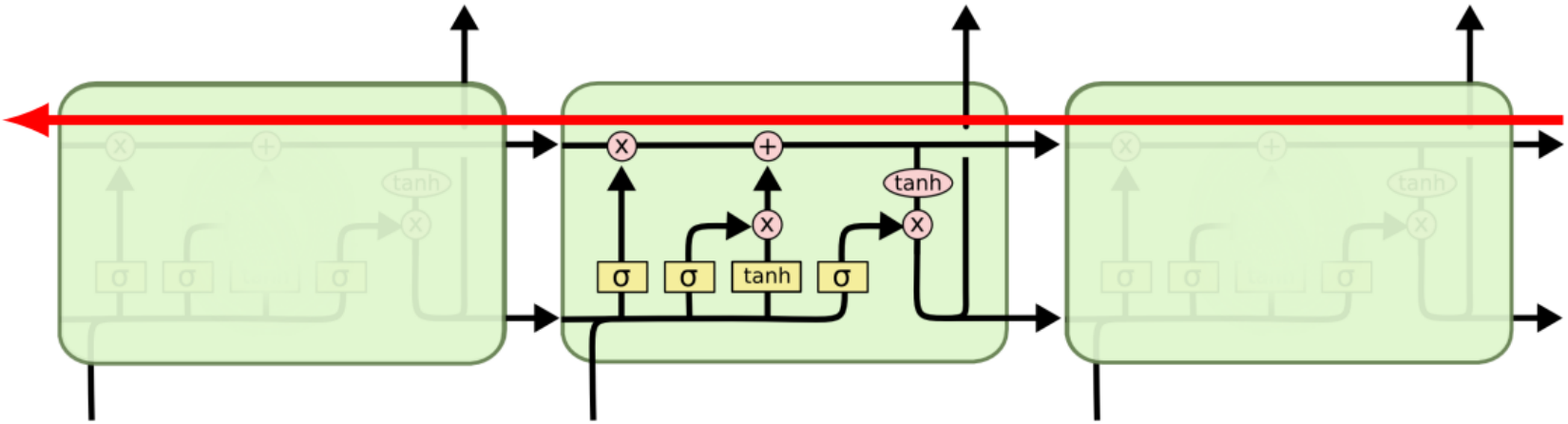
     c.f. vanilla RNN: same $\boldsymbol{W}$

     ⇒ more likely to have vanishing/exploding gradient

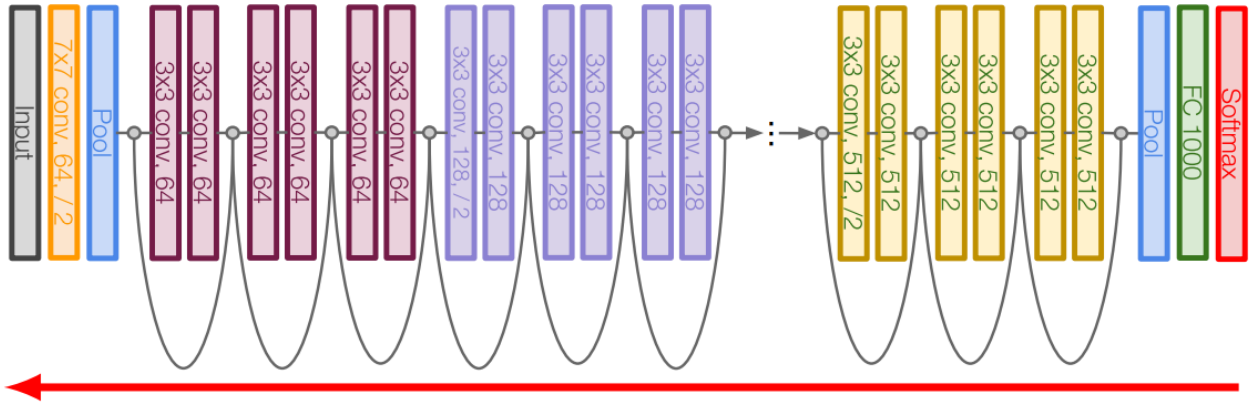


Gradient flow in LSTM

- Backpropagation through cell state: gradient super highway

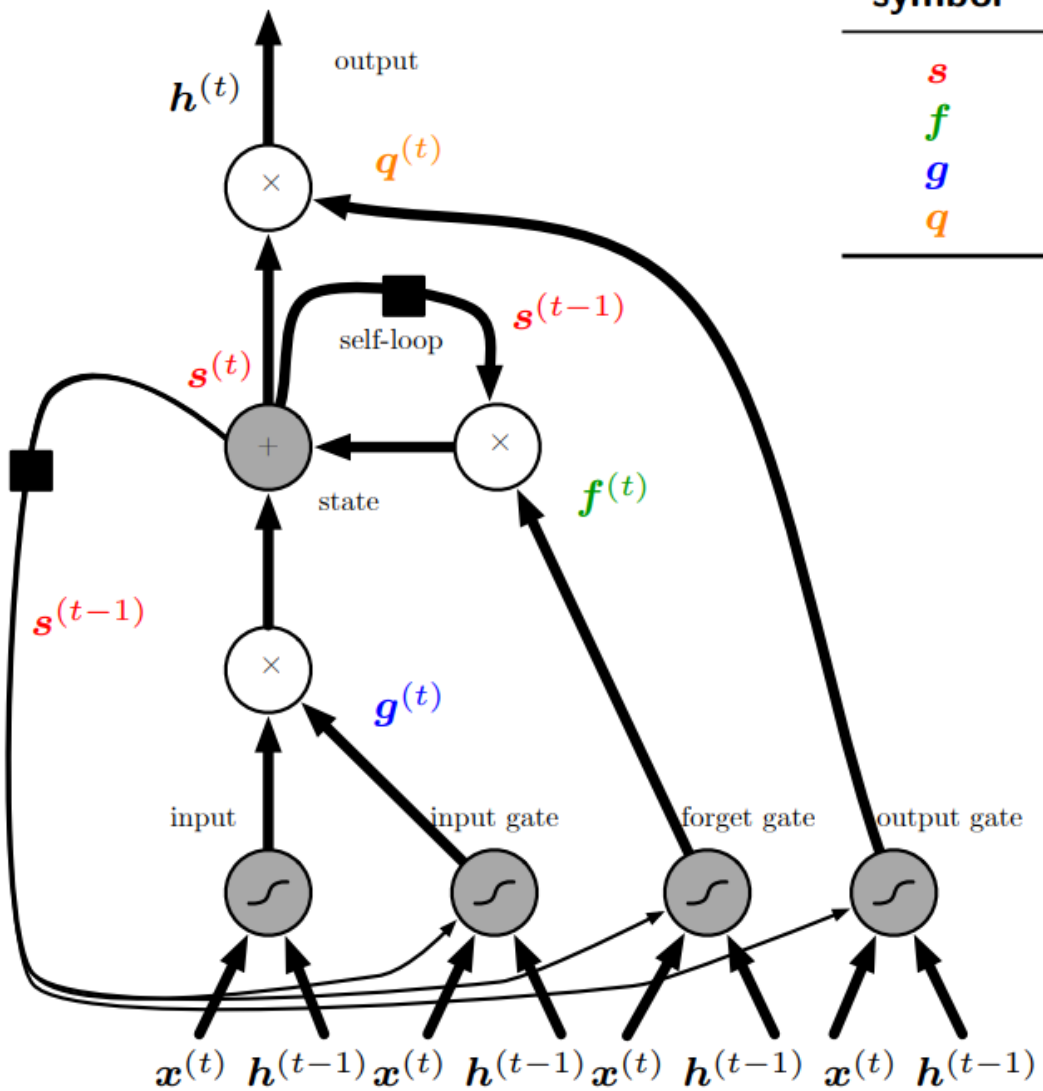


- Similar to the effect of skip connections in ResNet



Alternative presentations

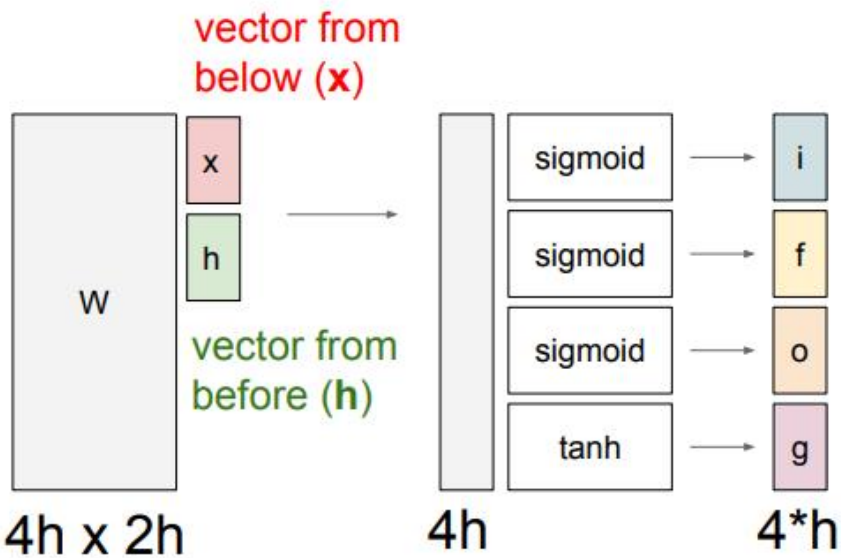
- textbook



symbol	role
s	cell state
f	forget gate
g	input gate
q	output gate

Alternative presentations

- Stanford CS231n



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

symbol	name	role
i	input gate	whether to write to cell
f	forget gate	whether to erase cell
o	output gate	how much to reveal cell
g	gate gate	how much to write to cell

Alternative presentations

- Stanford CS231n – Single-Layer RNNs

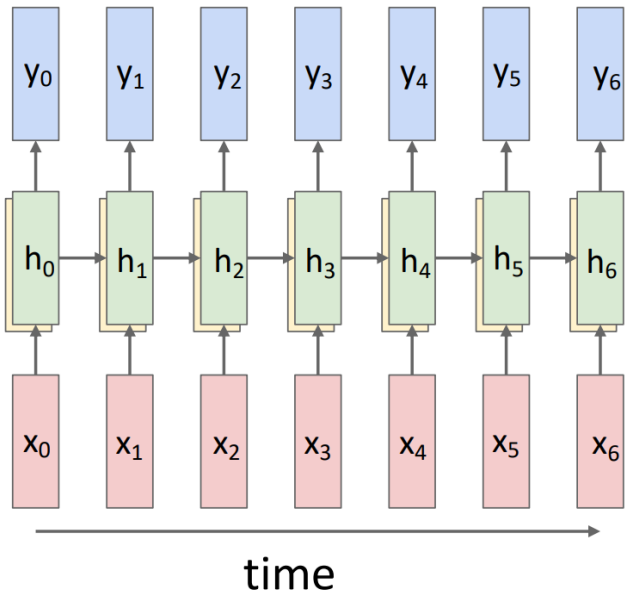
$$h_t = \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b_h\right)$$

LSTM:

$$\begin{pmatrix} i_t \\ f_t \\ o_t \\ g_t \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix}\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b_h\right)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$h_t = o_t \odot \tanh(c_t)$$



Alternative presentations

- Stanford CS231n – Multilayer RNNs

$$h_t^\ell = \tanh\left(W\begin{pmatrix} h_{t-1}^\ell \\ h_t^{\ell-1}\end{pmatrix} + b_h^\ell\right)$$

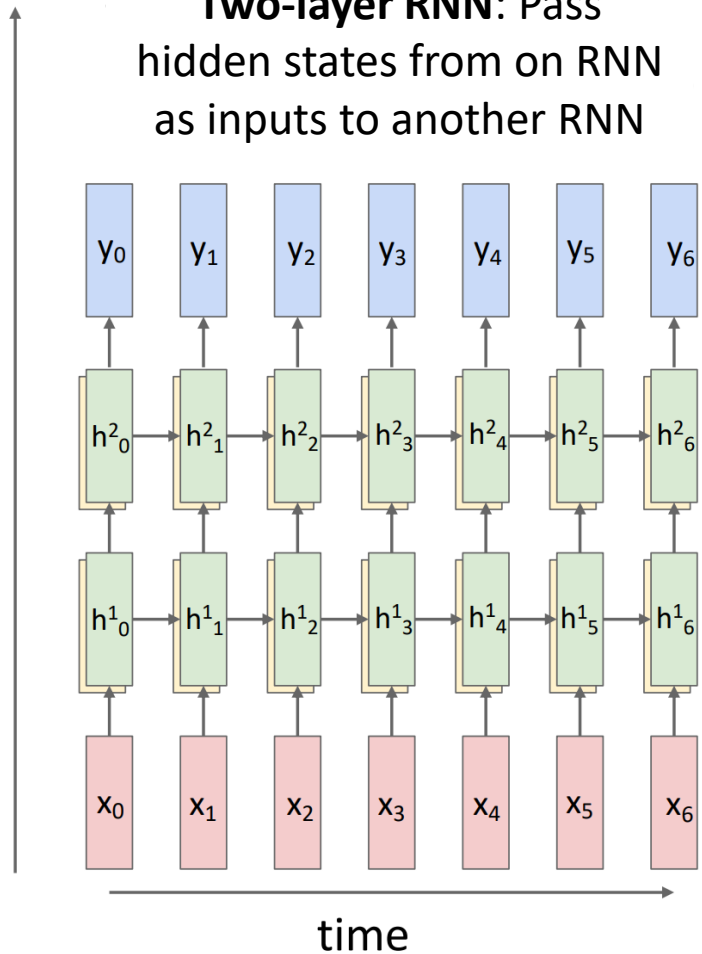
LSTM:

$$\begin{pmatrix} i_t^\ell \\ f_t^\ell \\ o_t^\ell \\ g_t^\ell \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix}\left(W\begin{pmatrix} h_{t-1}^\ell \\ h_t^{\ell-1}\end{pmatrix} + b_h^\ell\right)$$

$$c_t^\ell = f_t^\ell \odot c_{t-1}^\ell + i_t^\ell \odot g_t^\ell$$

$$h_t^\ell = o_t^\ell \odot \tanh(c_t^\ell)$$

Two-layer RNN: Pass hidden states from on RNN as inputs to another RNN



Alternative presentations

Pass hidden states from one RNN as inputs to another RNN

- Stanford CS231n – Multilayer RNNs

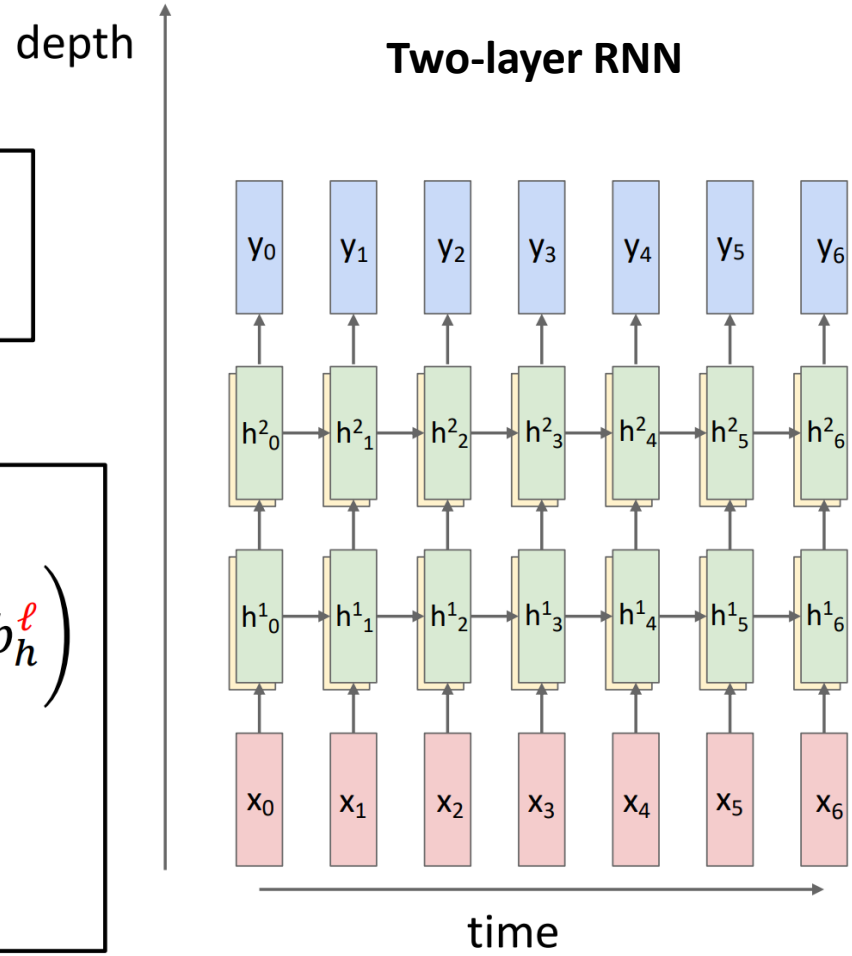
$$h_t^\ell = \tanh\left(W\begin{pmatrix} h_{t-1}^\ell \\ h_t^{\ell-1}\end{pmatrix} + b_h^\ell\right)$$

LSTM:

$$\begin{pmatrix} i_t^\ell \\ f_t^\ell \\ o_t^\ell \\ g_t^\ell \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix}\left(W\begin{pmatrix} h_{t-1}^\ell \\ h_t^{\ell-1}\end{pmatrix} + b_h^\ell\right)$$

$$c_t^\ell = f_t^\ell \odot c_{t-1}^\ell + i_t^\ell \odot g_t^\ell$$

$$h_t^\ell = o_t^\ell \odot \tanh(c_t^\ell)$$



Alternative presentations

Pass hidden states from one RNN as inputs to another RNN

- Stanford CS231n – Multilayer RNNs

$$h_t^\ell = \tanh\left(W\begin{pmatrix} h_{t-1}^\ell \\ h_t^{\ell-1}\end{pmatrix} + b_h^\ell\right)$$

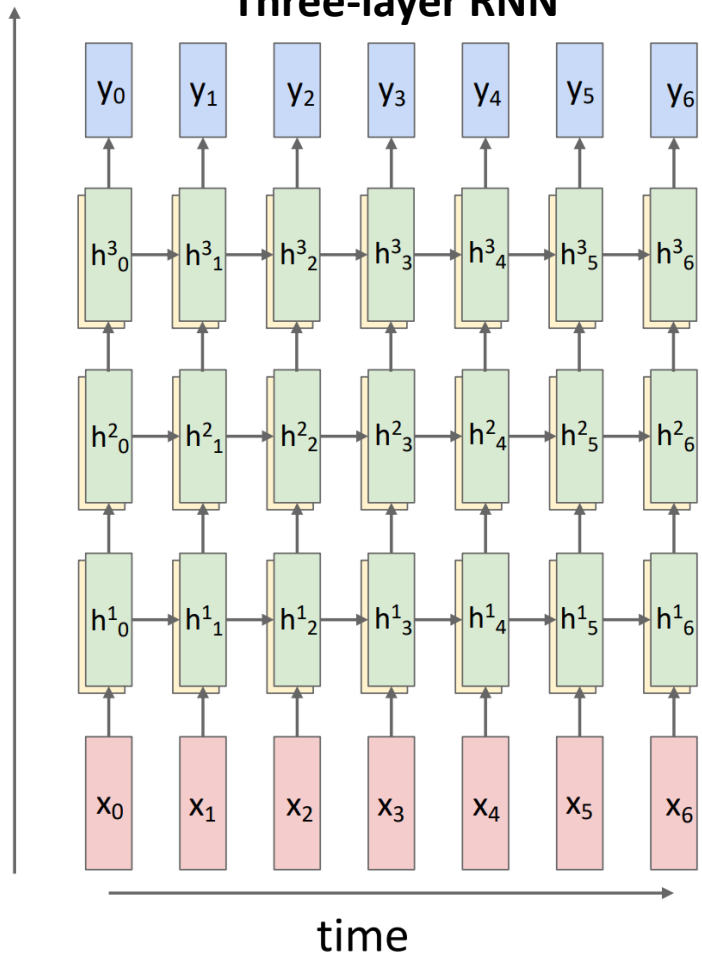
LSTM:

$$\begin{pmatrix} i_t^\ell \\ f_t^\ell \\ o_t^\ell \\ g_t^\ell \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix}\left(W\begin{pmatrix} h_{t-1}^\ell \\ h_t^{\ell-1}\end{pmatrix} + b_h^\ell\right)$$

$$c_t^\ell = f_t^\ell \odot c_{t-1}^\ell + i_t^\ell \odot g_t^\ell$$

$$h_t^\ell = o_t^\ell \odot \tanh(c_t^\ell)$$

Three-layer RNN



Outline

Challenge: Learning Long-Term Dependencies

Gated RNNs

- Long Short-Term Memory (LSTM)
- LSTM Variants

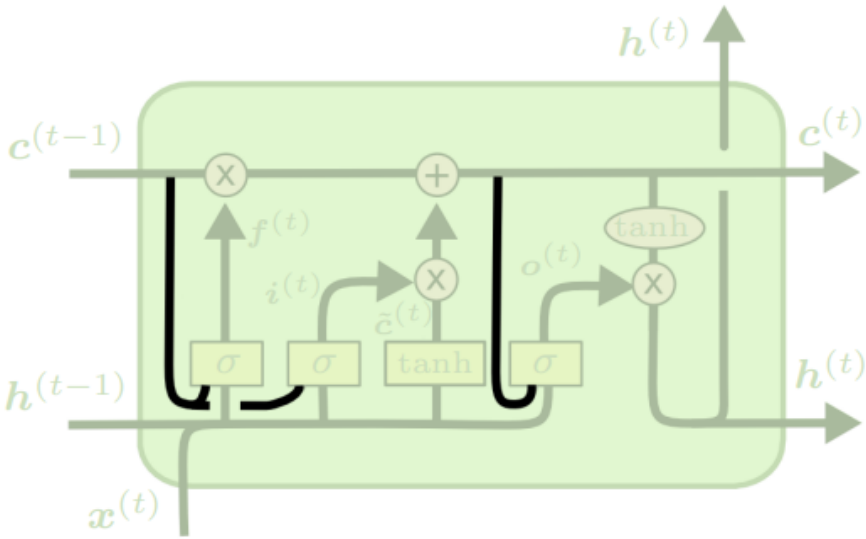
Summary

Peephole connections

- Make the gate layers look at cell state

  i.e. use cell state as extra input to gates

  - need additional parameters



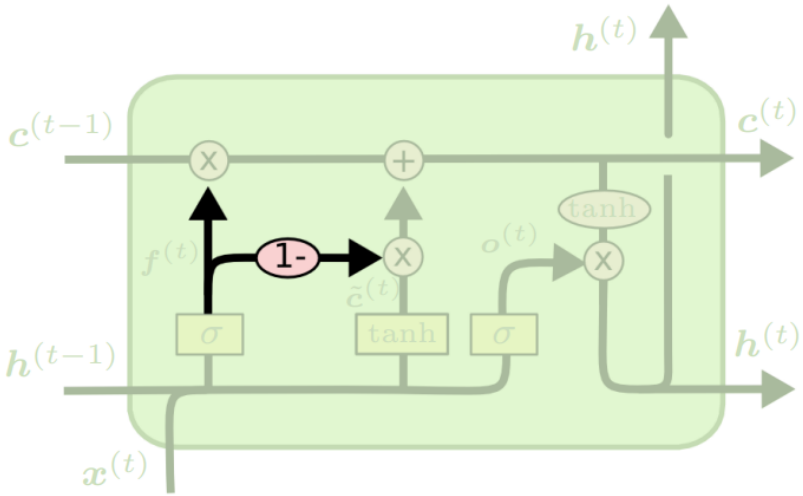
$$\boldsymbol{f}^{(t)} = \sigma(\boldsymbol{W}_f[\boldsymbol{c}^{(t-1)}, \boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_f)$$
$$\boldsymbol{i}^{(t)} = \sigma(\boldsymbol{W}_i[\boldsymbol{c}^{(t-1)}, \boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_i)$$
$$\boldsymbol{o}^{(t)} = \sigma(\boldsymbol{W}_o[\boldsymbol{c}^{(t)}, \boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}] + \boldsymbol{b}_o)$$

  - Actual peephole connections may vary from architecture to architecture

Coupling forget/input gates

- Previously: separate decisions about what to forget (by forget gate) and what to update (by input gate)

- Now: make these decisions together
  - How much to update = how much to not forget



- recall: cell state update

$$\underbrace{\boldsymbol{c}^{(t)}}_{\text{new state}} = \overbrace{\boldsymbol{f}^{(t)}}^{\text{how much to forget}} \odot \underbrace{\boldsymbol{c}^{(t-1)}}_{\text{old state}} + \overbrace{\boldsymbol{i}^{(t)}}^{\text{how much to update}} \odot \underbrace{\tilde{\boldsymbol{c}}^{(t)}}_{\text{new candidate values}}$$

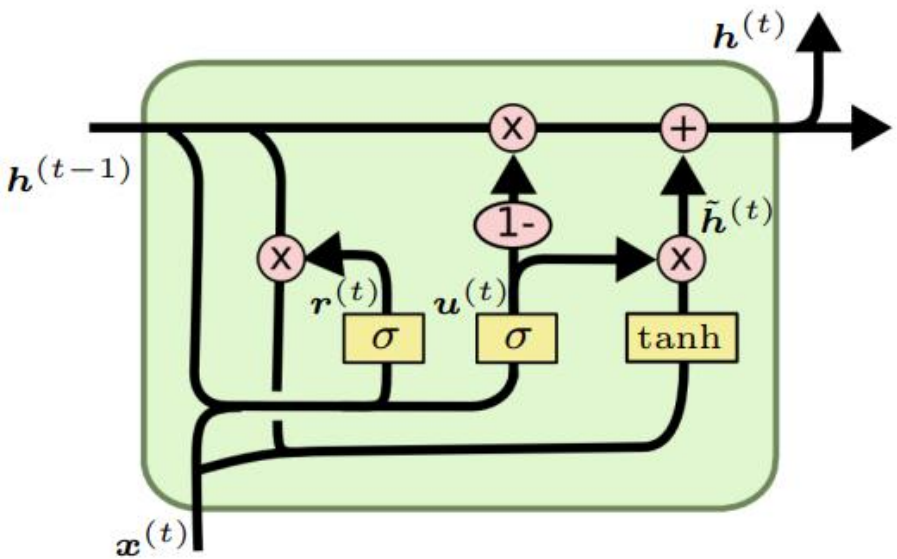
- coupled gating

$$\boldsymbol{c}^{(t)} = \boldsymbol{f}^{(t)} \odot \boldsymbol{c}^{(t-1)} + (1 - \boldsymbol{f}^{(t)}) \odot \tilde{\boldsymbol{c}}^{(t)}$$

Gated Recurrent Unit (GRU)

- Major changes over LSTM: simplification
  - Combines forget and input gates into a single "update gate"
  - Introduces "reset gate"
  - Merges cell state and hidden state into one state

- Potential advantages over LSTM
  - Fewer parameters ⇒ sometimes easier training/better performance



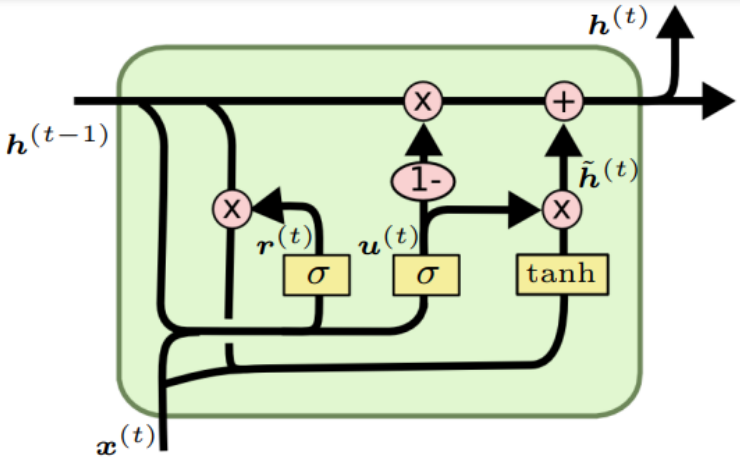
$$u^{(t)} = \sigma(W_u[h^{(t-1)}, x^{(t)}])$$

$$r^{(t)} = \sigma(W_r[h^{(t-1)}, x^{(t)}])$$

$$\tilde{h}^{(t)} = \tanh(W[r^{(t)} \odot h^{(t-1)}, x^{(t)}])$$

$$h^{(t)} = (1 - u^{(t)}) \odot h^{(t-1)} + u^{(t)} \odot \tilde{h}^{(t)}$$

Gated Recurrent Unit (GRU)



$$\boldsymbol{u}^{(t)} = \sigma(\boldsymbol{W}_u[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}]) \quad (1)$$

$$\boldsymbol{r}^{(t)} = \sigma(\boldsymbol{W}_r[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}]) \quad (2)$$

$$\tilde{\boldsymbol{h}}^{(t)} = \tanh(\boldsymbol{W}[\boldsymbol{r}^{(t)} \odot \boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}]) \quad (3)$$

$$\boldsymbol{h}^{(t)} = (1 - \boldsymbol{u}^{(t)}) \odot \boldsymbol{h}^{(t-1)} + \boldsymbol{u}^{(t)} \odot \tilde{\boldsymbol{h}}^{(t)} \quad (4)$$

eq	meaning
(1)	update gate $\boldsymbol{u}^{(t)}$ is learned from old state $\boldsymbol{h}^{(t-1)}$ and current input $\boldsymbol{x}^{(t)}$
(2)	reset gate $\boldsymbol{r}^{(t)}$ is learned from old state $\boldsymbol{h}^{(t-1)}$ and current input $\boldsymbol{x}^{(t)}$
(3)	• new candidate value $\tilde{\boldsymbol{h}}^{(t)}$ reflects old state $\boldsymbol{h}^{(t-1)}$ and current input $\boldsymbol{x}^{(t)}$ • reset gate $\boldsymbol{r}^{(t)}$ controls how much old state $\boldsymbol{h}^{(t-1)}$ is used • current input $\boldsymbol{x}^{(t)}$ is fully used regardless of $\boldsymbol{r}^{(t)}$
(4)	• current state $\boldsymbol{h}^{(t)}$ combines old state $\boldsymbol{h}^{(t-1)}$ and new candidate value $\tilde{\boldsymbol{h}}^{(t)}$ • update gate $\boldsymbol{u}^{(t)}$ controls how to mix these two

Variants and comparisons

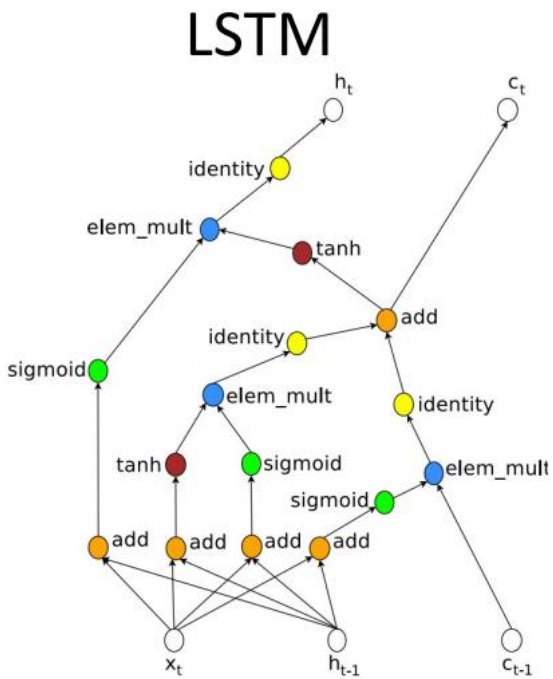
- Many more variants of LSTM can be designed

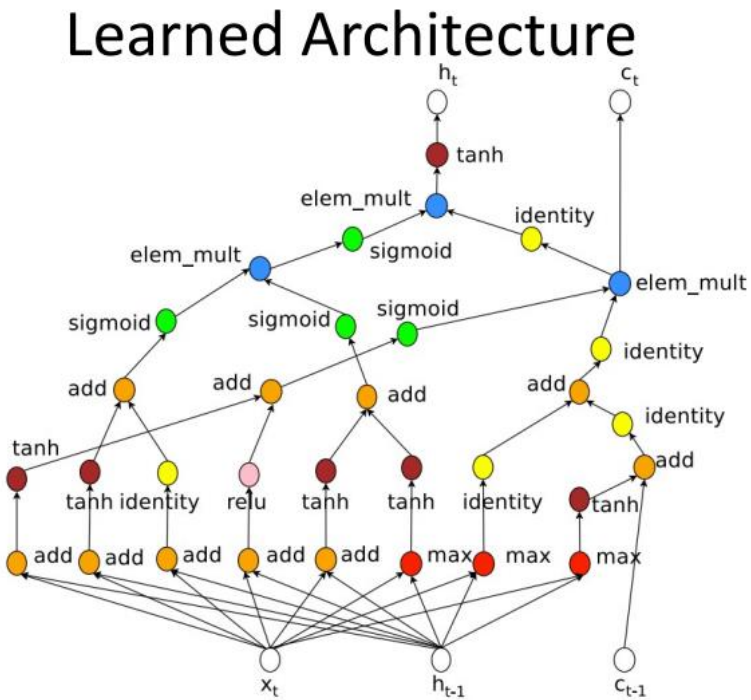
- However, several investigations found
  - No variant would clearly beat both of LSTM/GRU across a wide range of tasks (Greff et al., 2015; Jozefowicz et al., 2015)

- Greff et al. (2015) found:
  - A crucial ingredient is the forget gate

- Jozefowicz et al. (2015) found:
  - 10,000 architectures with evolutionary search
  - Adding a bias of 1 to LSTM forget gate
    - Makes LSTM as strong as the best of explored architectural variants
  - "An empirical exploration of recurrent network architectures", ICML 2015

Variants and comparisons: Neural Architecture Search

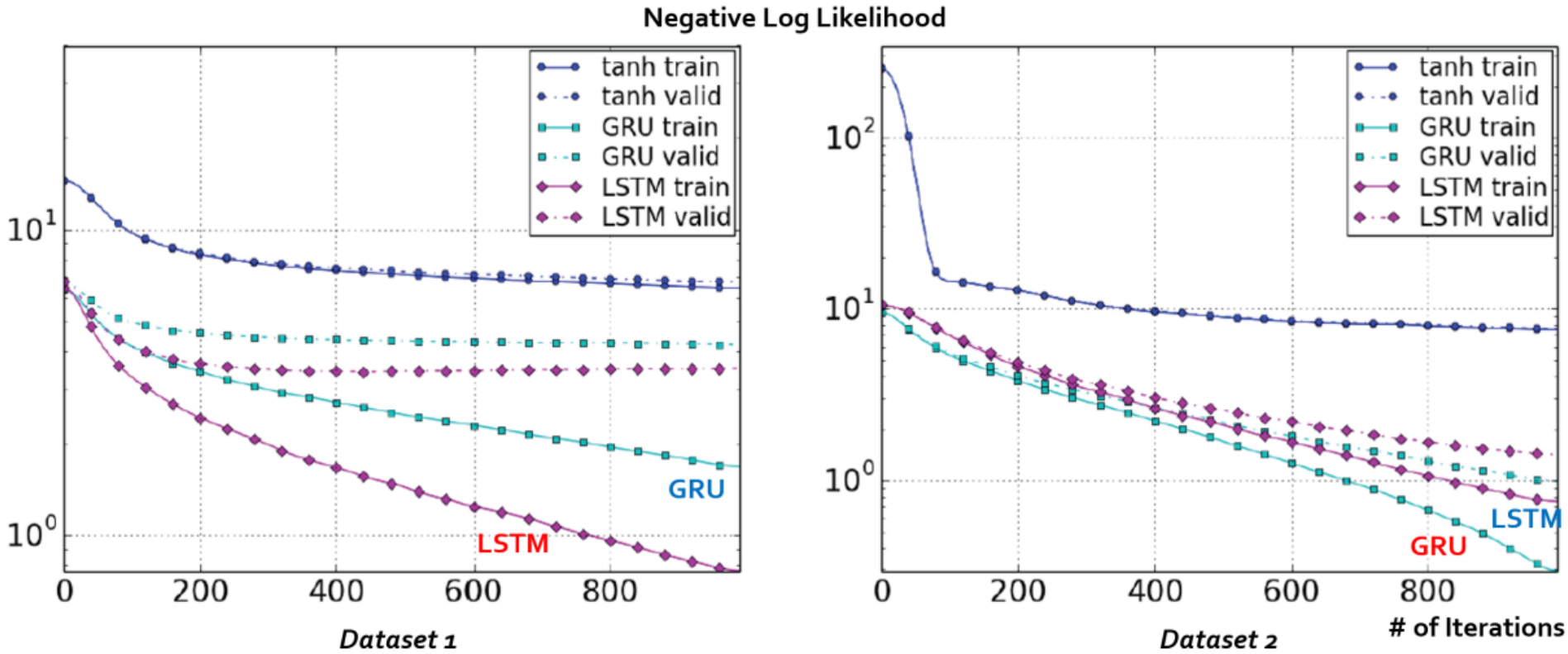




Zoph and Le, "Neural Architecture Search with Reinforcement Learning", ICLR 2017

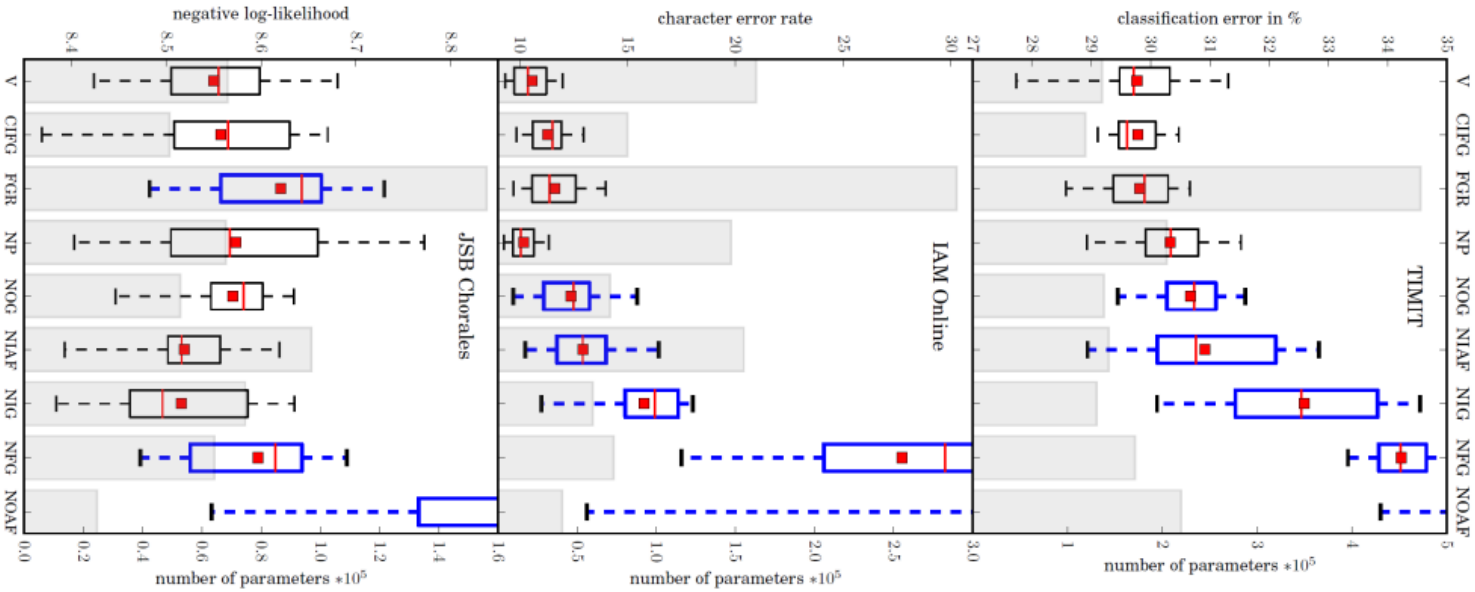
“No Clear Winner”

- But GRU/LSTM-RNNs certainly outperform traditional RNNs



LSTM: a search space odyssey

- Greff et al. 2015
- Large-scale analysis of eight LSTM variants
  - Speech/handwriting recognition, polyphonic music modeling
  - 5400 experimental runs (≈ 15 years of CPU time)
- Result: “no variants can improve on standard LSTM significantly”
  - Most critical: forget gate & output activation funciton



Toward Optimal RNN Architectures

- Jozefowicz et al. 2015
- Objectives
  - Determine whether LSTM is optimal or better one exists
  - Better understand the role of individual components
- Results from evaluating over 10,000 different architectures
  - Found an architecture that outperforms LSTM/GRU
  - Fine print: “on some but not all tasks”
- Another finding: adding a bias of one to LSTM forget gate
  - Closes the gap between LSTM and GRU

Outlook

- Memory-augmented neural net (MANN)
  - Memory networks: learn Q&A tasks
  - Neural Turing machines: learn algorithms

<u>"the fall of RNN/LSTM"</u>

- Critical problems of RNN/LSTM
  - Often very difficult to train
  - Hardware acceleration issue: memory-bandwidth bound

- (arguably better) alternatives
  - Temporal convolutional networks (TCN)
  - Pervasive attention (2D CNN for seq-to-seq prediction)
  - The Transformer (machine translation with self-attention; no RNN/CNN)
    - "attention is all you need"

Outline

Challenge: Learning Long-Term Dependencies

Gated RNNs

- Long Short-Term Memory (LSTM)
- LSTM Variants

Summary

Summary

- Challenge of training RNN: learning long-term dependency
  - main cause: vanishing gradient and/or exploding gradient
  - heuristic solutions: gradient clipping, IRNN (limited applicability)
  - generic solution requires changes in architecture ⇒ gated RNN

- gated RNN: LSTM, GRU
  - create paths through time without vanishing/exploding gradients
  - gates: control how much "open" the signal flows ($0 \le \sigma \le 1$)
  - cell state: stores system state like a counter ($-1 \le \tanh \le 1$)
  - backprop path through cell state: mainly element-wise multiplication

  ⇒ helpful to maintain strong gradient flows (similar to ResNet idea)

- further extensions and recent criticism
  - memory-augmented neural nets: learn Q&A tasks or algorithms
  - "attention is all you need"